

The second mechanism is the NTFS change journal. NTFS keeps a list of all the change records for directories and files on the volume in a special file, which programs can read using special file-system control operations, that is, the *FSCTL_QUERY_USN_JOURNAL* option to the *NtFsControlFile* API. The journal file is normally very large, and there is little likelihood that entries will be reused before they can be examined. However, if journal entries do get reused before an application can examine them, then the app just needs to enumerate the directory tree it is interested in, to sync up with its state. After that, it can resume using the journal.

File Encryption

Computers are used nowadays to store all kinds of sensitive data, including plans for corporate takeovers, tax information, and love letters, which the owners do not especially want revealed to anyone. Information loss can happen when a notebook computer is lost or stolen, a desktop system is rebooted using an MS-DOS floppy disk to bypass Windows security, or a hard disk is physically removed from one computer and installed on another one with an insecure operating system.

Windows addresses these problems by providing an option to encrypt files, so that even in the event the computer is stolen or rebooted using MS-DOS, the files will be unreadable. The normal way to use Windows encryption is to mark certain directories as encrypted, which causes all the files in them to be encrypted, and new files moved to them or created in them to be encrypted as well. The actual encryption and decryption are not managed by NTFS itself, but by a driver called **EFS (Encryption File System)**, which registers callbacks with NTFS.

EFS provides encryption for specific files and directories. There is also another encryption facility in Windows called **BitLocker** which runs as a block filter driver and encrypts almost all the data on a volume, which can help protect data no matter what—as long as the user takes advantage of the mechanisms available for strong keys. Given the number of systems that are lost or stolen all the time, and the great sensitivity to the issue of identity theft, making sure secrets are protected is very important. An amazing number of notebooks go missing every day. Major Wall Street companies supposedly average losing one notebook per week in taxicabs in New York City alone.

11.9 WINDOWS POWER MANAGEMENT

The **power manager** supervises power usage throughout the system. Historically management of power consumption consisted of shutting off the monitor display and stopping the disk drives from spinning. But the issue is rapidly becoming more complicated due to requirements for extending how long notebooks can run

on batteries, and energy-conservation concerns related to desktop computers being left on all the time and the high cost of supplying power to the huge server farms that exist today.

Newer power-management facilities include reducing the power consumption of components when the system is not in use by switching individual devices to standby states, or even powering them off completely using *soft* power switches. Multiprocessors shut down individual CPUs when they are not needed, and even the clock rates of the running CPUs can be adjusted downward to reduce power consumption. When a processor is idle, its power consumption is also reduced since it needs to do nothing except wait for an interrupt to occur.

On heterogeneous multiprocessor systems with multiple types of processors, significant power savings can be achieved by scheduling appropriate work to more efficient processors. The power manager closely collaborates with the kernel thread scheduler to influence its quality-of-service scheduling policies. For example, if the system is low on battery, the power manager can configure power policy such that all low-QoS threads exclusively get scheduled to efficiency cores.

Windows supports a special shut down mode called **hibernation**, which copies all of physical memory to disk and then shuts down the machine, reducing power consumption to zero. Because all the memory state is written to disk, you can even replace the battery on a notebook while it is hibernated. When the system resumes after hibernation, it restores the saved memory state (and reinitializes the I/O devices). This brings the computer back into the same state it was before hibernation, without having to login again and start up all the applications and services that were running. Windows optimizes this process by ignoring unmodified pages backed by disk already and compressing other memory pages to reduce the amount of I/O bandwidth required. The hibernation algorithm automatically tunes itself to balance between I/O and processor throughput. If there is more processor available, it uses expensive but more effective compression to reduce the I/O bandwidth needed. When I/O bandwidth is sufficient, hibernation will skip the compression altogether. With the current generation of multiprocessors, both hibernation and resume can be performed in a few seconds even on systems with many gigabytes of RAM.

An alternative to hibernation is **standby mode** where the power manager reduces the entire system to the lowest power state possible, using just enough power to refresh the dynamic RAM. Because memory does not need to be copied to disk, this is somewhat faster than hibernation on some systems.

Despite the availability of hibernation and standby, many users are still in the habit of shutting down their PC when they finish working. Windows uses hibernation to perform a pseudo shutdown and startup, called **HiberBoot**, that is much faster than normal shutdown and startup. When the user tells the system to shutdown, HiberBoot logs the user off and then hibernates the system at the point they would normally login again. Later, when the user turns the system on again, HiberBoot will resume the system at the login point. To the user it looks like startup was

very, very fast because most of the system initialization steps are skipped. Of course, sometimes the system needs to perform a real shutdown in order to fix a problem or install an update to the kernel. If the system is told to reboot rather than shutdown, the system undergoes a real shutdown and performs a normal boot.

On phones and tablets, as well as the newest generation of laptops, computing devices are expected to be always on yet consume little power. To provide this experience, modern Windows implements a special version of power management called **CS (Connected Standby)**. CS is possible on systems with special networking hardware which is able to listen for traffic on a small set of connections using much less power than if the CPU were running. A CS system always appears to be on, coming out of CS as soon as the screen is turned on by the user. Connected standby is different than the regular standby mode because a CS system will also come out of standby when it receives a packet on a monitored connection. Once the battery begins to run low, a CS system will go into the hibernation state to avoid completely exhausting the battery and perhaps losing user data.

Achieving good battery life requires more than just turning off the processor as often as possible. It is also important to keep the processor off as long as possible. The CS network hardware allows the processors to stay off until data have arrived, but other events can also cause the processors to be turned back on. In NT-based Windows device drivers, system services, and the applications themselves frequently run for no particular reason other than to *check on things*. Such *polling* activity is usually based on setting timers to periodically run code in the system or application. Timer-based polling can produce a cacophony of events turning on the processor. To avoid this, Windows requires that timers specify an imprecision parameter which allows the operating system to coalesce timer events and reduce the number of separate occasions one of the processors will have to be turned back on. Windows also formalizes the conditions under which an application that is not actively running can execute code in the background. Operations like checking for updates or freshening content cannot be performed solely by requesting to run when a timer expires. An application must defer to the operating system about when to run such background activities. For example, checking for updates might occur only once a day or at the next time the device is charging its battery. A set of system brokers provide a variety of conditions which can be used to limit when background activity is performed. If a background task needs to access a low-cost network or utilize a user's credentials, the brokers will not execute the task until the requisite conditions are present.

Many applications today are implemented with both local code and services in the cloud. Windows provides **WNS (Windows Notification Service)**, which allows third-party services to push notifications to a Windows device in CS without requiring the CS network hardware to specifically listen for packets from the third party's servers. WNS notifications can signal time-critical events, such as the arrival of a text message or a VoIP call. When a WNS packet arrives, the processor will have to be turned on to process it, but the ability of the CS network hardware

to discriminate between traffic from different connections means the processor does not have to awaken for every random packet that arrives at the network interface.

11.10 VIRTUALIZATION IN WINDOWS

In the early 2000s, as computers were getting larger and more powerful, the industry started turning to virtual machine technology to partition large machine into a number of smaller virtual machines sharing the same physical hardware. This technology was originally used primarily in data centers or hosting environments. In the next decade, however, attention turned to more fine-grained software virtualization and containers came into fashion.

Docker Inc. popularized the use of containers on Linux with its popular Docker container manager. Microsoft added support for these types of containers to Windows in Windows 10 and Windows Server 2016 and partnered with Docker Inc. so that customers could use the same popular management platform on Windows. Additionally, Windows started shipping with the Microsoft Hyper-V hypervisor so that the OS itself could leverage hardware virtualization to increase security. In this section, we will first look at Hyper-V and its implementation of hardware virtualization. Then we will study containers built purely from software and describe some of the OS features that leverage hardware virtualization features.

11.10.1 Hyper-V

Hyper-V is Microsoft's virtualization solution for creating and managing virtual machines. The hypervisor sits at the bottom of the Hyper-V software stack and provides the core hardware virtualization functionality. It is a Type-1 (bare metal) hypervisor that runs directly on top of the hardware. The hypervisor uses virtualization extensions supported by the CPU to virtualize the hardware such that multiple *guest* operating systems can run concurrently, each in its own isolated virtual machine, called a **partition**. The hypervisor works with the other Hyper-V components in the **virtualization stack** to provide virtual machine management (such as startup, shutdown, pause, resume, live migration, snapshots, and device support). The virtualization stack runs in a special privileged partition called the **root partition**. The root partition must be running Windows, but any operating system, such as Linux, can be running in guest partitions which are also called **child partitions**. While it is possible to run guest operating systems that are completely unaware of virtualization, performance will suffer. Nowadays, most operating systems are **enlightened** to run as a guest and include guest counterparts to the root virtualization stack components which help provide higher-performance paravirtualized disk or network I/O. An overview of Hyper-V components is given in Fig. 11-50. We will discuss these components in the upcoming sections.